SQL in Application Code

- Two main integration approaches
  - (SL) Statement level interface → embedded SQL
    - embed SQL →
  - (CL) Call-level interface → JDBC
    - special API to → ODBC
      SQL commands

do not need to know schema version

		Precompiler	Schema Info	data source info	multiple DBC	efficiency	SQL Injection Risk
SL	static	YES	YES	YES	No	higher	lower
	dynamic	YES	NO			lower	higher
CL		No	No	No	Yes	lower	statement higher / prepared statement lower

Schema of DB must be known at time program is written for embedded SQL

→ can get info at runtime instead of compile time

- SQLSTATE - common error vars defined here

```
EXEC SQL SELECT S.sname, S.age
INTO :c_sname, :c_age
FROM sailors S
WHERE S.sid = :c_sid
```

out parameters

in parameter

- for db, start index from 1

Project 2

- need to use LIKE for partial matching
- cross-check?
- program should NOT crash
- use gradle for simplicity (cuz that's what I know best)

- Impedance Mismatch problem
  - fixed with cursor

OPEN: when SQL statement is executed — at runtime

FETCH: when it moves to the next row

INSENSITIVE cursor: cannot make changes to DB with cursor

If I want to update result, then declare sensitive cursor

→ fixes issue with collecting set of tuples

CLOSE: closes the cursor

Cursor can be reopened when in variable is different

Note the poll question!!

- execute Update()
  only returns number of rows (affected)
  ↳ so use executeQuery() instead
- see the comparison table for
- JDBC vs SQLJ
  ↳ for managing complexity better

when to choose what?

L13: Functional Dependencies
Attribute Closure

L14: Normal Forms
Normalize

L15: Properties of Decomposition
Lossless Join + DP
Minimal Cover

L16: Decomposition Algorithms

- to make any changes, I need to declare a sensitive cursor
- embedded SQL recommended in systems that require high performance without JDBC overhead
- for dynamic SQL, we cannot reuse, since we need to prepare

JDBC/ODBC

JDBC does not require knowing db info or schema info

- can connect to multiple DBs with multiple separate drives
- all interactions with the DB performed via a connection object

Prepared Statement is more efficient to use since its made to reuse as it is precompiled

- ? are placeholders

- Keyword → Redundancy
- FD: most important kind of constraint in the relational model
  - associated with the schema, not the instance

- can I represent key constraint using FD?

$X \rightarrow Y$

X functionally determines Y

alternatively,
Y is dependent on X

PhoneBook

Phone #	Name	City
7035550101	Alice	Fairfax
7035550101	Alice	Fairfax

if I know someone's phone no., can I always figure out their name

Phone # → Name

- look out for redundancies in data instances

$t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$

t_1 and t_2 → Any two rows in your table

$t_1[X]$ → look at the X columns of row 1

$t_1[X] = t_2[X]$ → Row 1 and Row 2 have same X values

⇒ → forces

$t_1[Y] = t_2[Y]$ → Row 1 and Row 2 must also have same Y value

---

FD between non-key attributes is a RED FLAG that data is being repeatedly unnecessarily

- FD means one attr's value is completely controlled by another
  - when controlling attr appears in multiple rows, its dependent value gets copy pasted → i.e. redundancy

- why repetition is problematic
  - update anomaly
  - insertion anomaly
  - deletion anomaly

She is going way too fast for me to observe...

didn't understand this

- see explanation in PPX

- k = {$A_1, A_2, \ldots A_n$} is a candidate key for relation R if

K → All other attributes of R
AND
No other subset of K → all other attributes of R

- $X \rightarrow YZ$ $\begin{cases} X \rightarrow Y \\ X \rightarrow Z \end{cases}$
  ↳ LHS is the determinant

cant decompose LHS

$\cancel{XY \rightarrow YZ}$ $\cancel{X \rightarrow Z}$ $\cancel{XY \rightarrow Z}$

- Closure of Attributes
  - find all attributes dependent on {A,B}?

if I know value of attrs X, what else can I figure out?

- i.e. closure of X
  AKA X^+

- think yourself as a detective
  - start with one clue, let it lead to more clues

- for example:
  R = (A, B, C, D, E, G)
  FDs: $AB \to C$, $BC \to AD$, $D \to E$, $CE \to B$
  ∴ $\{A, B\}^+$?

- build a snowball...

Round		Snowball
Start	A and B	$\{A, B\}$
1	$AB \to C$	$\{A, B, C\}$
2	$BC \to AD$	$\{A, B, C, D\}$
3	$D \to E$	$\{A, B, C, E\}$
4	X	Done

∴ $\{A, B\}^+ = \{A, B, C, D, E\}$
G was never reached

- what about $\{C, D\}^+$
  $\{C, D\}^+ \Rightarrow \{C, D\}$
  $\xrightarrow{D \to E} \{C, D, E\}$
  $\xrightarrow{CE \to B} \{B, C, D, E\}$
  $\xrightarrow{BC \to A} \{A, B, C, D, E\}$
  G still missing
  ∴ $\{C, D\}$ is NOT a candidate key
→ $\{C, D, G\}^+ = \{A, B, C, D, E, G\}$
  ↳ is a superkey since it covers all attributes in R
  ↳ is a candidate key too since is minimal — removing G breaks the S.K property
  same applies for:
  $\{A, B, G\}^+$

Normalization
- 1NF: disallows multivalued attributes, composite attrs, and their combinations
- Prime vs. Nonprime Attrs
  ↳ number of any candidate key
  ↳ not a prime attr. (not a member of any C.K)
  disjoint
- Full FD vs. Partial FD
  ↳ removing a single attr breaks the dependency
  ↳ removing some attr and dep still holds
  e.g. EMP_PROJ(SSN, Pnumber, Hours, Ename)
  ∴ $\{SSN, Pnumber\} \to Hours$: full FD (need both to determine hours worked)
  $\{SSN, Pnumber\} \to Ename$: partial FD (SSN alone determines Ename)

- 2NF (Second Normal form)
  ↳ already in 1NF
  ↳ every nonprime attr is fully FD on primary key
  ∴ for 1NF → 2NF
  - find every partial dependency
  - move attr and partial key to new, separate relation
  as partial dependencies cause redundancies
  e.g. EMP_PROJ(SSN, Pnumber, Hours, Ename, Pname, Plocation)
  FD1: $\{SSN, Pnumber\} \to$ Hours (full)
  FD2: SSN → Ename (partial)
  - ename depends only on SSN
  FD3: Pnumber → Pname, Plocation (partial)
  - they depend only on Pnumber.

∴ After 2NF → 3 tables
  EP1 (SSN, Pnumber, Hours) - full dep
  EP2 (SSN, Ename) - employee details
  EP3 (Pnumber, Pname, Plocation) - project details
- 3NF (Third Normal form)
  - transitive dependency
    e.g. $A \to B$, $B \to C$
    ∴ C is transitively dependent on A via B
  - already 2NF
  - no nonprime attr is transitively dependent on P.K
  For 2NF → 3NF
  - find transitive deps
  - move TD attrs to new relation along with copy of intermediate determinant (B)

e.g.
EMP_DEPT (Ename, SSN, Bdate, Addr, Dnumber, Dname, Dmgr_SSN)
FD chain: SSN → Dnumber
  Dnumber → Dname, Dmgr_SSN
  (creates T.D)
∴ ED1 (Ename, SSN, Bdate, Address, Dnumber)
  ED2 (Dnumber, Dname, Dmgr_SSN)

- determinant: look at F.D set → cos of F.D
- to determine if LHS is superkey
  - compute attribute closure

- BCNF (Boyce-Codd Normal form)
  - stricter version of 3NF
  when I normalize to 3NF does it already normalize to BCNF?
  - if BCNF, no redundancies
  e.g. for $X \to Y$
    X is the determinant (the thing doing the "determining")
  - for every $X \to Y$ in R, X must be superkey (uniquely identify every row)
- vs. 3NF
  - 3NF says non-superkey determinant is okay as long as what it determines (Y) is part of some C.K (i.e. prime attr)

Ex. 1
⇒ Prime Attrs
  - Property#
  - County-name
  - Lot#
  Non prime Attrs
  - Area
  - Price
  - Tax-rate
- already 1NF
- for 1NF → 2NF
  FD2: PID → all other attr
  FD3: County-name → Tax-rate
  FD4: Area → Price
  ∴ 2NF
  ① PID#, CN, Lot#, Area, Price
  ② CN, Tax-rate
  TD
  ↳ dependent on PID#

- for 2NF → 3NF
  - Area, Price are NP
  ∴ ③ Area, Price
  ① PID#, County-name, Lot#, Area
- verifying BCNF (W)

Ex. 2 : P#, D, L, Pa, Co, S#, Sn, CR
  Prime
  - P#
  - D
  - L
  - CR
  Nonprime
  - Pa
  - Co
  - S#
  - Sn
  } required for 1NF
- already 1NF

- for 1NF → 2NF

FD2 : P# → Pa

both NP, so TD on P.K

partial dep → 2NF violation (not the full {P#, Idate})

FD3 : S# → Sn → T.D

FD4 : {Id, S#} → CR

∴ ① {P#, Pa}

② {P#, Id, It, Co, S#, Sn, CR}

- for 2NF → 3NF

③ {S#, Sn}

② {P#, Id, It, Co, S#, CR}

- for 3NF → BCNF

{Id, S#} → not a superkey

∴ BCNF violation

↳ since RHS is P

3NF only protects NP attrs

Needs to be included for CK

{A, C, D}⁺ ⇒ {A, B, C, D}

⇒ {A, B, C, D, E}

{B, C, D}⁺ ⇒ {B, C, D, E}

⇒ {A, B, C, D, E}

{C, D, E}

b) All attrs are prime

∴ already in 3NF

c) No

A → B

↳ not a superkey

BC → E

↳ not

ED → A not

④ {Id, S#, CR}

② {P#, Id, It, Co, S#}

	3NF	BCNF
FD: A → B, B is prime, A not a superkey	Allowed	Violation
FD: A → B, B is nonprime, A is not a superkey	Violation	Violation

P P NP NP

A B C D

2NF ✓

(requires partial check for subset of prime attributes)

3NF ✗

1. Consider a relation R with five attributes ABCDE. You are given the following dependencies: A→B, BC→E, and ED→A.
   a) List all candidate keys for R.
   b) Is R in 3NF?
   c) Is R in BCNF?

2. Suppose that we have the following three tuples in a legal instance of a relation schema S with three attributes ABC (listed in order): (1, 2, 3), (4, 2, 3), and (5, 3, 3).
   a) Which of the following dependencies can you infer does not hold over schema S? A→B, BC→A, B→C
   b) Can you identify any dependencies that hold over S?

3. Suppose you are given a relation R with four attributes, ABCD. For the following sets of FDs: C→D, C→A, B→C, assuming those are the only dependencies that hold for R.
   a) Identify the candidate key(s) for R.
   b) Identify the best normal form that R satisfies (1NF, 2NF, 3NF, or BCNF).

① R {A, B, C, D, E}

FD1 : A → B

FD2 : BC → E

FD3 : ED → A

a) Utilize Attr. Closure

↳ RHS: {A, B, E}

∴ not included

{C, D}⁺